



SoundMap

RELAZIONE DEL PROGETTO “YOUR CITY IS LISTENING TO”

CORSO DI LABORATORIO DI APPLICAZIONI MOBILI – A.A 2023/24

Realizzato da:

- **Nome e Cognome:** Matilde Zoccolillo
- **Email:** matilde.zoccolillo@studio.unibo.it
- **Matricola:** 0001021318

Sommario

OVERVIEW DEL PROGETTO	2
DETTAGLI DI IMPLEMENTAZIONE.....	3
STACK TECNOLOGICO UTILIZZATO	3
ARCHITETTURA	4
CICLO DI VITA DELL' APP	4
GESTIONE DEI PERMESSI	4
COMPONENTI PRINCIPALI.....	5
GESTIONE DEL TOKEN	10
IMPLEMENTAZIONE DEI REQUISITI	11
<i>Use cases</i>	11
FUNZIONALITÀ AGGIUNTIVE	15
CONCLUSIONI	17

Overview del Progetto

L'applicazione Android **Your City is Listening To** è progettata per raccogliere e condividere dati musicali all'interno di una città, utilizzando funzionalità di registrazione audio, geolocalizzazione e visualizzazione su mappa. Gli utenti possono registrare file audio tramite il microfono dello smartphone, geo-taggarle le registrazioni e caricarle su un back-end remoto per l'elaborazione. Le informazioni sulle registrazioni, una volta elaborate, sono rese disponibili su una mappa interattiva.

L'utente, una volta autenticato, può registrare audio tramite il microfono del proprio dispositivo, riascoltarlo e caricarlo sul server tramite connessione dati o wi-fi.

Tutte le registrazioni sono visibili come markers sulla mappa interattiva. Cliccando sul marker è possibile vedere le informazioni della registrazione, quali:

- autore,
- posizione,
- coordinate,
- genere,
- mood,
- bpm,
- danceability.

Tramite la sezione “le mie registrazioni”, inoltre, l'utente può riascoltare le proprie registrazioni salvate in locale, anche senza connessione, ed eventualmente eliminarle o nasconderele.

Infine, nella sezione “statistiche” è possibile vedere le statistiche calcolate sulle registrazioni dell'utente.

Dettagli di Implementazione

Stack tecnologico utilizzato

Versioni Android:

- Compile SDK: 34
- Min SDK: 21
- Target SDK: 34

L'applicazione è sviluppata al 100% in nativo (linguaggio **Kotlin**) tramite **Android Studio**.

Debugging: il debugging è stato svolto con l'emulatore fornito dall'editor ed uno smartphone Samsung (Galaxy S9 – Android 10) collegato via cavo al computer. I logs erano visibili su **Logcat** e permettevano un debugging mirato.

Data management: per comunicare con il server è stato scelto **Retrofit**: una libreria HTTP per Android che semplifica la comunicazione con API RESTful, offrendo una gestione efficiente e tipizzata delle richieste HTTP. Nella creazione delle richieste http, è stato d'aiuto l'utilizzo di **Postman**.

Per il salvataggio dei dati in locale, invece, è stato scelto **Room**: una libreria di persistenza per Android che fornisce un livello di astrazione sopra SQLite, semplificando l'archiviazione e la gestione di dati strutturati.

Interfaccia Utente (Ui): per la creazione dell'interfaccia, è stato integrato nel progetto **Jetpack Compose**: un toolkit moderno che ha permesso di definire interfacce reattive in modo dichiarativo. Questo approccio ha migliorato leggibilità, manutenibilità e garantito uno stile pulito e moderno.

Multithreading: Per la gestione asincrona e non bloccante delle operazioni di rete e database, sono state utilizzate **coroutines**, unità di calcolo suspendibili che possono essere eseguite in modo concorrente con il resto del codice. È stato specificato il **Coroutine Context**, utilizzando dispatcher come Dispatchers.IO o Dispatchers.Main, per garantire che le operazioni di I/O non interferissero con il thread principale non venissero svolte sul main thread e non bloccassero l'esecuzione principale dell'app.

Tasks in background: per permettere all'utente di posticipare l'upload dell'audio è stato implementato **WorkManager**: un'API per programmare compiti asincroni e differibili che persistono anche dopo il riavvio del dispositivo.

Questo approccio allo sviluppo ha garantito che l'app fosse scalabile, performante e facile da mantenere, adottando strumenti moderni e ottimizzati per il ciclo di vita di applicazioni Android.

Architettura

L'app si basa su un'architettura **MVVM** per separare la logica di business dall'interfaccia utente, migliorando la manutenibilità e la testabilità. Le principali componenti includono:

- **Model:** Gestisce l'accesso ai dati tramite Room per il database locale e Retrofit per la comunicazione con il back-end.
- **ViewModel:** Conserva lo stato dell'interfaccia utente e interagisce con i repository.
- **View:** Jetpack Compose è utilizzato per creare un'interfaccia moderna e reattiva.

La comunicazione tra view e viewModel avviene tramite oggetti osservabili e variabili che ne rappresentano lo stato nel composabile. Le variabili osservabili sono mutabili e private nel viewModel e immutabili nei composables. Nei composabile vengono implementati **effects** (come LaunchedEffect o DisposableEffect) e strutture di controllo (if, else, when) per reagire ai cambiamenti di stato. Le variabili di stato utilizzano *remember* o *rememberSavable* per essere associate a tutto il ciclo di vita del composabile o sopravvivere a cambiamenti di stato come la rotazione dello schermo.

Ciascuna repository viene creata tramite *factory* per garantire un'inizializzazione controllata e una gestione coerente delle dipendenze.

Ciclo di vita dell'app

Essendo l'app stata sviluppata con architettura MVVM ed interfaccia Compose non è stato necessario utilizzare intensivamente strumenti tradizionali come **Activity** e **Intent**, né gestire stati del ciclo di vita come onPause, onResume o onDestroy. In Compose, il ciclo di vita è strettamente legato alla gestione dello **stato**. Gli strumenti principali sono remember, rememberSavable, liveData e stateFlow, di cui invece è stato fatto uso abbondante.

Per la navigazione è stato utilizzato **Jetpack Compose Navigation**, per cui ogni destinazione è rappresentata da un composabile. Lo stato di navigazione è gestito tramite il **NavController**, che permette di navigare tra le destinazioni, conservare lo stato del back stack e sincronizzare gli eventi del ciclo di vita con le composizioni.

Gestione dei Permessi

Nel file AndroidManifest.xml sono dichiarati i permessi necessari:

- **POST_NOTIFICATIONS:** Per inviare notifiche.
- **RECORD_AUDIO:** Per registrare l'audio.
- **ACCESS_FINE_LOCATION:** Per la geolocalizzazione precisa.
- **INTERNET** e **ACCESS_NETWORK_STATE:** Per comunicare con il back-end.

Componenti Principali

(core/app/)

App.kt

La classe **App** è registrata come `android:name` nell'elemento `<application>` dell'`AndroidManifest.xml`. Questo segnala al sistema Android che questa classe deve essere usata come punto di avvio per l'intera applicazione.

```
<application
    android:name=".core.app.App"
```

Questa classe definisce il punto di ingresso per la configurazione globale dell'applicazione Android.

Estende *Application* e viene utilizzata per configurare risorse condivise, come database, repository, e servizi, che saranno accessibili da altre parti dell'applicazione.

All'avvio dell'app, infatti, viene eseguito il metodo `onCreate()` che configura le risorse:

- Configura **Flogger** per i logs
- Crea i canali di notifica
- Inizializza le risorse che sono condivise dalle componenti dell'app, in modo da averne un'unica istanza globale.

La classe è strutturata con Singleton pattern tramite il companion object, permettendo di accedere alle risorse globali da qualsiasi punto dell'applicazione.

MainActivity.kt

Analogamente ad `App.kt`, questa classe è definita come activity principale (entry point) nel manifest tramite l'intent filter.

```
<activity
    android:name=".features.splash.SplashActivity"
    android:exported="true"
    android:theme="@style/Theme.LAM01.Splash">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

Utilizza la funzione `setContent` per avviare l'interfaccia grafica basata su Jetpack Compose. Configura la navigazione attraverso `AppNavHost`, che collega le diverse schermate dell'app.

(core/navigation/)

AppNavHost.kt

La classe AppNavHost è il componente centrale del sistema di navigazione dell'applicazione, implementato utilizzando **Jetpack Compose Navigation**.

La classe definisce il **grafo di navigazione** dell'applicazione, specificando le schermate principali (destinazioni). Gestisce inoltre l'inizializzazione di componenti chiave, come i ViewModel, e configura il GlobalExceptionHandler.

(core/util)

NotificationHelper.kt

La classe NotificationHelper è progettata per centralizzare la gestione delle notifiche nell'applicazione.

(core/util/exceptions/)

GlobalExceptionHandler.kt e AuthenticationException.kt

Classi create per gestire nello specifico il caso di eccezioni legate all'autenticazione rimandando l'utente al login.

(core/util/player/)-(core/util/recorder/)

AndoridAudioPlayer.kt e AndroidAudioRecorder.kt

Queste classi implementano la logica per registrare e riprodurre file audio estendendo le classi MediaRecorder e MediaPlayer di Android. Sono progettate per essere più robuste rispetto all'uso diretto di MediaPlayer/Recorder, grazie a un'accurata configurazione e gestione delle eccezioni e delle risorse.

(core/work)

AudioUploadCorountineWorker.kt

Questa classe è una sottoclasse di **CoroutineWorker**: una classe fornita da WorkManager in Android che consente di eseguire operazioni asincrone usando Coroutines di Kotlin. La classe è progettata per gestire il caricamento asincrono dei file audio. È stata creata per permettere l'upload differito dell'audio, facendolo avvenire in automatico appena il dispositivo è connesso ad una rete mobile o wi-fi. Una volta effettuato l'upload, l'utente viene avvisato grazie ad una notifica apposita.

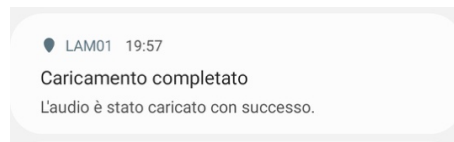


Figura 1: notifica di upload avvenuto

WorkScheduler.kt

WorkScheduler viene richiamato per pianificare l'upload dell'audio tramite AudioUploadCorountineWorker.kt e la funzione scheduleAudioUpload(params). Tra i params, oltre i valori relativi all'audio, viene specificato un valore booleano "requireWifi" che, se true, aggiunge alla workrequest il requisito (constraint) di connessione wi fi.

(data/local/)

TokenManager.kt

TokenManager fornisce una gestione centralizzata per il ciclo di vita del token di autenticazione. In essa viene salvato il token e l'orario di scadenza. Tramite il metodo isTokenExpired() è possibile controllare la validità del token ed eventualmente cancellare il token in istanza.

PreferencesHelper.kt

La classe PreferencesHelper fornisce un'interfaccia per gestire dati persistenti tramite **SharedPreferences**, una funzionalità nativa di Android per memorizzare valori chiave-valore sul disco. Questa classe è fondamentale per salvare e recuperare informazioni essenziali quali:

- Username
- Password
- Id
- Token e scadenza del token

Grazie al salvataggio su preferenceHelper, è possibile mantenere in memoria il token anche dopo la chiusura dell'app e far sì che l'utente non debba rieffettuare il login se non necessario.

AppDatabase.kt

La classe AppDatabase rappresenta il database locale dell'applicazione, implementato utilizzando **Room**, una libreria di persistenza di Android.

Il metodo `getDatabase(context)` implementa il *pattern singleton* per garantire che esista una sola istanza di `AppDatabase` nell'app ed utilizza il metodo `Room.databaseBuilder()` per creare il database. Supporta, inoltre, migrazioni tra versioni del database (`addMigrations`).

AudioDao.kt

`AudioDao` è un'interfaccia annotata con `@Dao` (data access object), che definisce le query per interagire con la tabella audio.

AudioEntity.kt

La data class `audioEntity` definisce le caratteristiche dell'entità audio.

(data/remote/)

TokenService.kt

La classe `TokenService` ha il compito di gestire il recupero e la validazione del token di autenticazione per le richieste al server. La funzione `getValidToken()` della classe controlla che ci sia un token salvato nel `TokenManager` ed in caso contrario ne ricava uno mandando la richiesta all'endpoint specifico, evitando di reindirizzare l'utente al login.

UserSessionManager.kt

Classe creata per centralizzare i metodi da eseguire a seguito di un `logout` o `deleteAccount` rispettando il principio di responsabilità singola. In questo modo, tutte le dipendenze necessarie al corretto `logout` o eliminazione dell'account sono iniettate solo in `userSessionManager`. Qui vengono dunque ripuliti i dati relativi all'account.

(data/remote/api/)

RetrofitInstance.kt

La classe `RetrofitInstance` è progettata per fornire un'implementazione singleton di `Retrofit`. Questo approccio garantisce che ci sia una sola istanza di `Retrofit` e delle relative configurazioni nell'intera applicazione, riducendo il consumo di risorse e semplificando la gestione del client. **`RetrofitInstance`** viene istanziata **lazily**, ossia solo quando la proprietà `api` o un'altra risorsa del singleton viene utilizzata per la prima volta. Questo è reso possibile dalla parola chiave `by lazy`, che ritarda l'inizializzazione fino al momento del bisogno.

ApiService.kt

ApiService funge da interfaccia per definire i metodi che corrispondono agli endpoint del server remoto. Ogni metodo è annotato con specifiche annotazioni HTTP (@GET, @POST, ecc.), che indicano il tipo di richiesta e l'URL dell'endpoint.

(data/remote/models/)

In questo package sono definite tutti i dataclass relativi alle richieste e alle risposte del server.

(data/repository/)

In questo package sono definite le repositories che hanno la responsabilità di comunicare con il server o con il database. Le repositories sono definite in base all'entità di cui si occupano: utenti e audio. Una terza repository (mapRepository) è stata creata in un secondo momento per alleggerire le dipendenze concentrarsi sulla responsabilità relativa alla gestione dei markers.

(features/)

In questa directory sono presenti i file kt relativi all'interfaccia suddivisi nei seguenti packages: audio, auth, map, splash e stats. Ogni package contiene a sua volta file contenenti le funzioni composabile per la creazione dinamica delle interfacce ed viewmodels di ciascuna di esse. A seguire degli esempi.

MainMapScreen.kt *(features/map/ui)*

Questa funzione è particolarmente importante perché rappresenta la schermata principale dell'app. Da qui vengono richiamate le composables relative alla registrazione audio, all'upload e alle informazioni sui brani. È poi possibile navigare verso il menù di navigazione.

SplashActivity.kt *(features/splash/)*

La classe SplashActivity implementa una schermata di avvio personalizzata (Splash Screen) per l'applicazione, mostrandone il logo. Utilizza il metodo *postDelayed* per ritardare di due secondi il passaggio alla schermata principale (**MainActivity**). Dopo il ritardo, viene avviata un'intent esplicita per avviare MainActivity e chiudere la SplashActivity con *finish()*.

Gestione del token

Il token di autenticazione, una volta ottenuto, viene salvato tra le preferenze condivise dell'app sul dispositivo. All'avvio dell'app, inoltre, viene istanziata la classe *TokenManager* descritta in precedenza. Durante l'inizializzazione, viene recuperato l'eventuale token salvato ed il relativo orario di scadenza (il token ha validità di un'ora) e viene avviato un job con timer. Allo scadere del timer il token viene ripulito sia da tokenManager che dalle shared preferences. Qualora l'utente volesse rieseguire una richiesta che richiede l'autenticazione, il token verrà eventualmente rinnovato tramite i metodi *executeAutheticatedRequest* delle repository, che si rifanno a *getValidToken()* di *TokenService*. Qui, se il token risulta null, viene rieffettuata l'API request per ottenere un nuovo token valido.

Implementazione dei requisiti

Use cases

I requisiti del progetto sono traducibili nei seguenti use-cases:

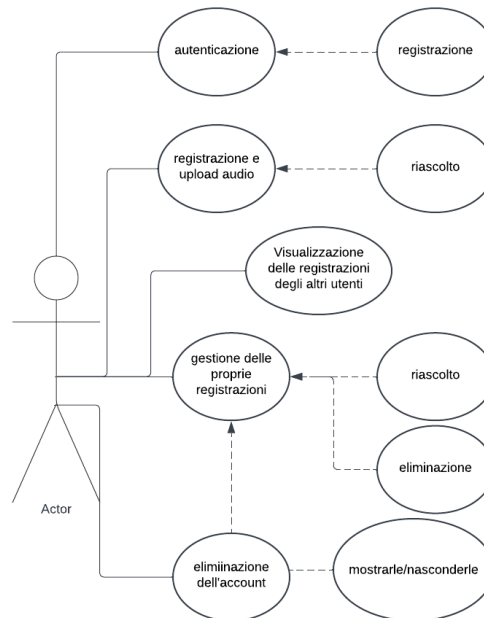


Figura 2: Use-case diagram del sistema

Ciascun use case è stato implementato come segue:

1- AUTENTICAZIONE

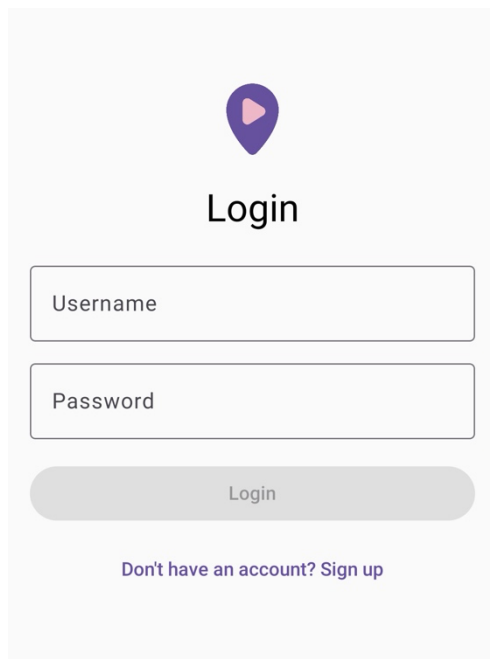
a. Login:

La schermata login è generata tramite la funzione composabile LoginScreen, la quale mostra i *textfields* per l'inserimento delle credenziali e salva i valori come variabili di stato savable (in modo che esse vengano ricordate durante il ciclo di vita del composabile). Al click sul bottone di accesso viene richiamata la funzione *login()* del viewModel associato alla view (authViewModel), che manda la richiesta di accesso al server tramite l'AuthRepository. Se il login avviene con successo, la variabile di stato authstate viene settata su "success" e il composabile reagisce reindirizzando alla schermata principale. In caso contrario viene mostrato un messaggio all'utente.

In caso di errore durante l'autenticazione (es. credenziali errate), un messaggio di errore viene mostrato all'utente attraverso una variabile di stato gestita nel ViewModel, garantendo un'interazione fluida e reattiva.

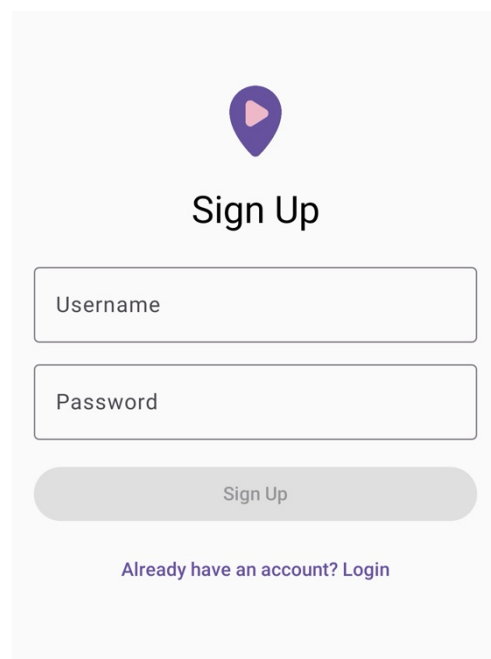
b. Registrazione:

funziona in maniera completamente analoga al login.



The Login screen features a purple location pin icon with a play button inside at the top center. Below it is the word "Login" in a large, bold, black font. There are two white input fields with thin black borders: the first is labeled "Username" and the second is labeled "Password". Below these fields is a wide, rounded grey button with the text "Login" in a smaller black font. At the bottom, there is a link that says "Don't have an account? Sign up" in a purple font.

Figura 5: Login screen



The Sign Up screen features a purple location pin icon with a play button inside at the top center. Below it is the word "Sign Up" in a large, bold, black font. There are two white input fields with thin black borders: the first is labeled "Username" and the second is labeled "Password". Below these fields is a wide, rounded grey button with the text "Sign Up" in a smaller black font. At the bottom, there is a link that says "Already have an account? Login" in a purple font.

Figura 3: Sign Up screen



Figura 4: Main map screen

2- REGISTRAZIONE E UPLOAD AUDIO

a. Registrazione audio:

L'utente può registrare il suono che lo circonda tramite il button “aggiungi una registrazione” del main map screen. Questo aprirà un AudioRecordingDialog a cui viene passata l'istanza dell'AudioViewModel.

Qui vengono eventualmente richiesti i permessi per lettura e scrittura su file locali ed il permesso per la registrazione audio tramite il microfono del dispositivo.

Durante la registrazione, viene creato e scritto un file salvato nel filesDir locale del dispositivo. Il file viene salvato come mp4 a causa della configurazione dell'audioRecorder e viene poi convertito in mp3, rendendolo riascoltabile dall'utente tramite l'audioPlayer. A questo punto l'utente può decidere se scartare l'audio, ritentare la registrazione o procedere con l'upload.

b. Upload audio:

Cliccando su “Conferma e invia” viene controllato se il dispositivo è connesso ad internet. Nel caso di connessione mobile, viene mostrato un dialog che permette di posticipare l'upload a quando il dispositivo sarà connesso tramite wi-fi. Nel caso di assenza completa di connessione, l'utente potrà scegliere di attendere il wi-fi o la connessione dati.

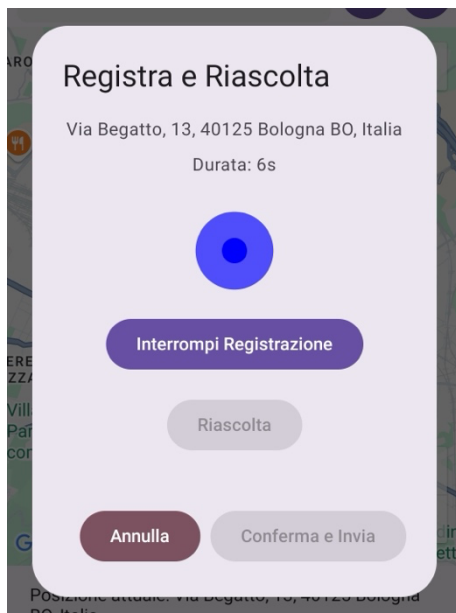


Figura 7: audio recording dialog

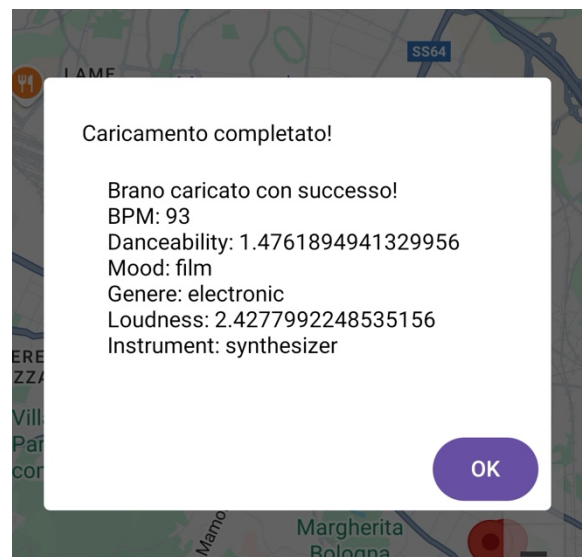


Figura 6: audio info dialog

3- VISUALIZZAZIONE DELLE REGISTRAZIONI DEGLI UTENTI

Tramite la `mainMapScreen`, l'utente può cliccare sui markers caricati dinamicamente sulla mappa in base alla sua posizione (in modo da evitare overhead dovuto al caricamento di markers non necessari). Ciascun oggetto `MapMarker` ha informazioni relative alla posizione e all'id dell'audio a cui fa riferimento. Cliccando sul marker, vengono *fetchate* e mostrate all'utente le informazioni dell'audio in un bottom sheet come illustrato nella figura 8.

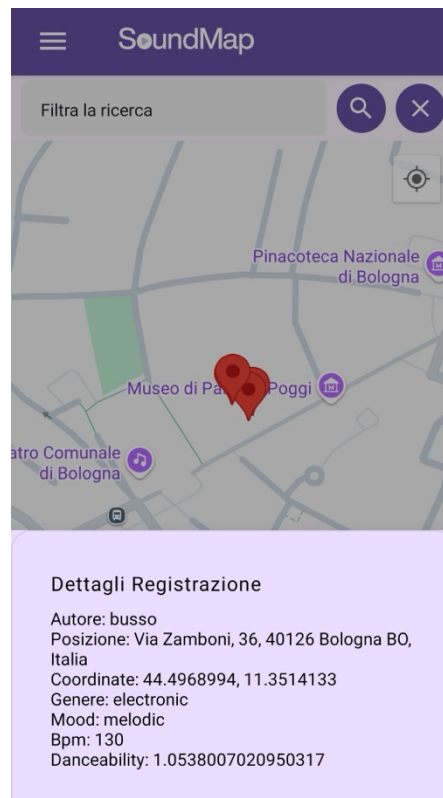


Figura 8: `AudioInfoBottomSheet`

4- GESTIONE DELLE PROPRIE REGISTRAZIONI

Tramite la sezione "Le mie registrazioni" l'utente può vedere tutte le registrazioni che ha caricato. Le informazioni relative alle proprie registrazioni sono ottenute dalla combinazione di informazioni locali (tags, nome della posizione e path al file audio) e remote (hidden). Da qui l'utente può riascoltare l'audio, rivedere i dati ad esso relativi e decidere se nascondere/mostrarlo oppure eliminarlo. Ciascuna delle precedenti azioni è implementata tramite un button che richiama un metodo specifico dell'`audioViewModel`, che a sua volta richiama l'`audioRepository`.

5- ELIMINAZIONE DELL'ACCOUNT

Infondo alla schermata del menù (`menuContent`) l'utente trova i buttons per effettuare il logout o l'eliminazione dell'account.

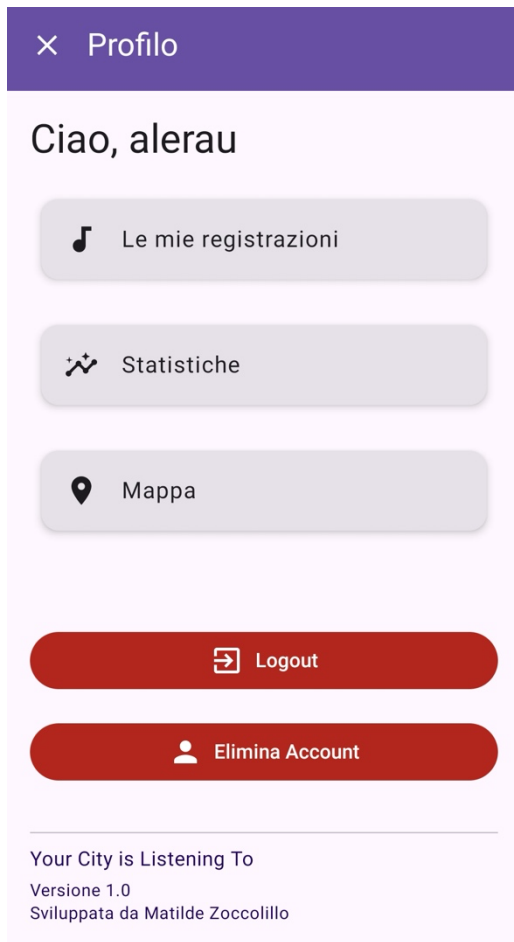


Figura 9: menù content



Figura 10: User recordings screen

Funzionalità aggiuntive

- VISUALIZZAZIONE DELLE STATISTICHE

L'utente, cliccando su "statistiche", viene reindirizzato allo StatsScreen a cui è passata l'istanza dello statisticsViewModel. Qui è possibile vedere le statistiche relative alle registrazioni caricate dall'utente. Questi valori sono calcolati direttamente tramite query specifiche fatte tramite l'audioDao. I valori vengono quindi ottenuti e salvati in istanza come dati osservabili nel viewModel.

- FILTRI SU MAPPA

L'utente può filtrare la ricerca dei markers tramite parole chiave che possono essere contenute nel genere, nel mood, nello strumento o nell'username dell'utente che ha caricato la registrazione.

Il filtro viene applicato iterando solo i markers visibili sullo schermo in quel momento. I marker risultanti sono poi sovrascritti sulla variabile LiveData "shownMarkers", osservata dal composabile della mappa, che contiene tutti i markers che devono essere mostrati sulla mappa

in quel momento. Se l'utente sposta la visualizzazione sulla mappa durante la ricerca, il filtro viene riapplicato sui nuovi markers.



Figura 13: StatsScreen



Figura 12: Filtro sui markers

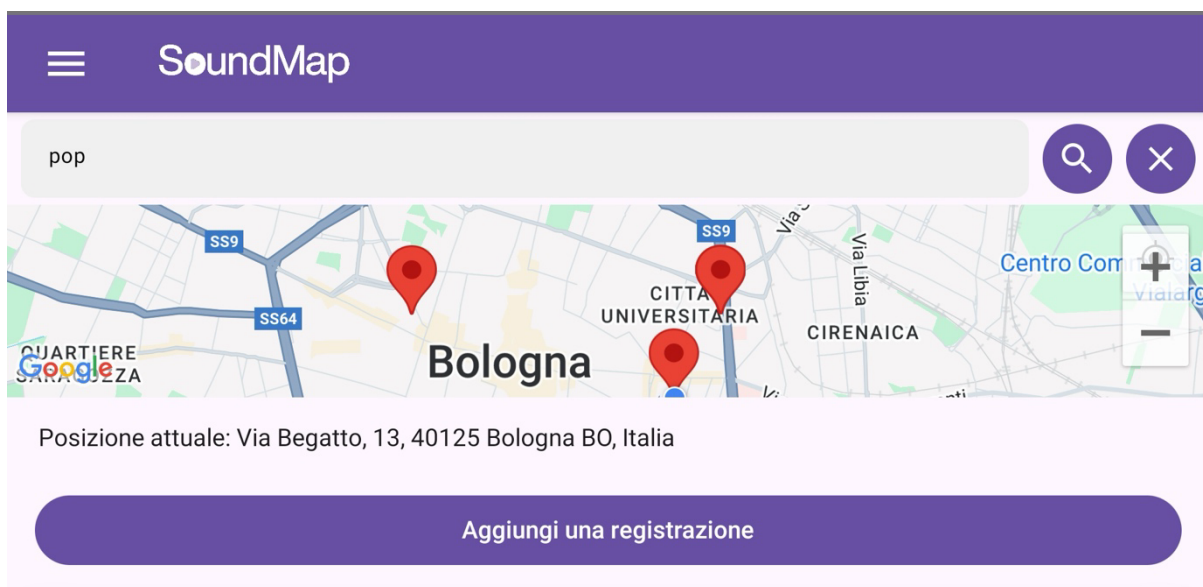


Figura 11: risultato del filtro dopo la rotazione dello schermo

Conclusioni

Lo sviluppo di **Your City is Listening To** ha rappresentato un'opportunità per mettere in pratica le best practice dello sviluppo Android e per integrare tecnologie moderne in un progetto innovativo. L'applicazione è stata progettata per essere scalabile, performante e semplice da mantenere, rappresentando un solido punto di partenza per ulteriori miglioramenti.

Gli sviluppi futuri potrebbero includere l'integrazione di un sistema di suggerimenti basato su algoritmi di machine learning per suggerire registrazioni in base alle preferenze degli utenti, il supporto per la localizzazione multilingua, ed un' integrazione con i social media per favorire la condivisione dei contenuti.